

ORIGINAL ARTICLE

Smart navigation system for emergency vehicles (SNSEV): utilizing fog and cloud computing technology for real-time traffic management

Fatma M. Talaat^{1,2}  · Samah A. Gamel³

Received: 11 January 2024 / Accepted: 9 April 2025 / Published online: 26 May 2025

© The Author(s) 2025

Abstract

The navigation of emergency vehicles is a critical component of effective emergency response in a smart city. To improve response time, it is necessary to have a navigation system that can predict the shortest path to the destination and adjust in real time to current traffic conditions. This paper proposes a smart navigation system for emergency vehicles (SNSEV), a real-time navigation algorithm for emergency vehicles in smart cities that utilize fog and cloud computing technology for real-time traffic management. IoT devices such as sensors and cameras collect real-time data on traffic conditions and roadblocks, which is then processed and analyzed using fog computing technology. Cloud computing technology is then utilized to provide emergency vehicles with real-time navigation and priority control, reducing response time and ensuring that they reach their destination as quickly as possible. This paper presents the proposed SNSEV algorithm and the system architecture and discusses its potential benefits and challenges. The results show that SNSEV can significantly improve the efficiency and effectiveness of emergency response in a smart city, leading to improved public safety and well-being. According to the findings of several experiments, SNSEV works better than its competitors because it allows for the highest possible throughput, the lowest possible bandwidth usage, and the shortest possible delay.

Keywords Real-time navigation · Emergency vehicles · Cloud computing · Traffic management

1 Introduction

The rapid growth of the population has led to an increase in demand for smart city development. However, the development of smart cities faces numerous challenges that require collaboration between the public and private sectors to find long-term solutions. One major challenge facing smart cities is traffic management, which has become a global concern due to the increase in traffic density and population expansion over the past few decades. Traffic congestion has led to numerous problems such as wasted time, property damage, pollution, and loss of life due to accidents. Moreover, delays in the delivery of vital medical equipment, essential drugs, and patients have also been reported [1].

Traffic lights play a crucial role in traffic management, but the duration of the green light and sequencing path must be carefully considered. Unfortunately, many traffic systems still use fixed or sequence light period techniques that are not suitable for dynamic traffic conditions [2]. Most countries use Green Light Sequences, which do not consider emergency vehicles such as police cars, ambulances, fire engines, etc. This situation can result in emergency vehicles waiting, leading to a loss of life or property. Moreover, as the vehicle population rises, the



response time of emergency vehicles, which is the duration between when an emergency service provider is called and when an emergency vehicle arrives, also increases. Reducing the response time by just one minute can increase the survival rate by 24% for patients with heart failure [3].

In order to decrease the negative impacts of traffic congestion, smart cities are adopting Information and Communication Technologies (ICT)-based solutions, particularly in smart mobility services. These solutions rely on wireless technologies such as sensor devices, fog, and cloud services to predict the shortest path from an emergency vehicle to its destination and control traffic lights along the way. This ensures that there are no disturbances along the road [4].

The proposed smart navigation system for emergency vehicles represents a significant advancement in traffic management technology. This innovative approach utilizes both fog and cloud computing to provide real-time traffic updates and route suggestions for emergency responders. The system is designed to incorporate a variety of data sources, including traffic cameras, and GPS tracking, to provide accurate and up-to-date traffic information to emergency personnel.

In addition to optimizing response times, the system also aims to reduce traffic congestion and improve overall safety on the roads. By providing real-time traffic updates and route suggestions, emergency responders can avoid congested areas and take alternate routes to reach their destinations more quickly and efficiently. This not only improves response times but also reduces the potential for accidents and other hazards that can occur when emergency vehicles are stuck in traffic or navigating crowded streets. Overall, this smart city navigation system has the potential to revolutionize emergency services in urban areas and improve the quality of life for residents by promoting safer, more efficient traffic flow.

The use of fog and cloud computing technology in this navigation system offers several key benefits over traditional traffic management systems. First and foremost, these technologies allow for faster processing of large amounts of data, which is critical for providing real-time traffic updates and route suggestions. Additionally, fog computing provides local processing capabilities, allowing the system to operate even in the event of internet connectivity issues or other disruptions. This ensures that emergency responders always have access to the information they need to make informed decisions about their routes.

Cloud computing, on the other hand, offers scalability and flexibility, allowing the system to adapt to changing traffic conditions and other factors that may impact emergency response times. Furthermore, cloud computing allows for secure data storage and sharing, enabling emergency responders to collaborate and communicate more effectively during emergencies. Ultimately, the combination of fog and cloud computing technology in this smart city navigation system provides a powerful tool for managing traffic in urban areas and improving emergency response times, making it an invaluable asset for cities looking to enhance their emergency services and promote safer, more efficient communities.

In this paper, we propose a methodology that utilizes wireless technologies to predict the shortest path for emergency vehicles and control traffic lights along the path to reduce response times and improve the survival rate of patients. The proposed methodology builds on the existing research and presents a unique solution that can be implemented in smart cities to enhance emergency services. A brief overview highlighting the contributions are:

- **Enhanced real-time decision-making:** SNSEV utilizes a combination of fog and cloud computing to process data closer to the source, enabling faster decision-making compared to traditional cloud-only systems.
- **Dynamic traffic management:** The integration of IoT devices allows for real-time data collection and analysis, enabling SNSEV to adaptively manage traffic lights and predict the shortest routes based on current conditions, which is often lacking in existing systems.
- **Prioritized navigation for emergency vehicles:** By incorporating a priority control mechanism, SNSEV ensures that emergency vehicles receive optimal routing that minimizes delays and enhances response times, surpassing standard navigation algorithms that do not account for emergency scenarios.

- **Robustness to network variability:** The architecture of SNSEV is designed to maintain functionality during network disruptions, which is a significant improvement over existing systems that may rely solely on stable cloud connectivity.

Novelty and contribution

The primary novelty of our work lies in the development of the **SNSEV (smart navigation system for emergency vehicles)** algorithm, which leverages real-time data from IoT devices in conjunction with fog and cloud computing technologies. This approach enables the efficient routing of emergency vehicles in urban environments by addressing critical real-time navigation challenges that have not been fully resolved by prior routing algorithms. Below are the key contributions that distinguish our work from existing research:

- i. **Integration of fog and cloud computing:** Unlike conventional routing strategies that either rely solely on cloud computing or traditional methods, SNSEV integrates both fog and cloud computing for more rapid and localized decision-making. Fog computing allows for real-time data processing close to the source, reducing latency in emergency vehicle routing decisions, while cloud computing provides the infrastructure for large-scale traffic analysis and routing optimization.
- ii. **Real-time data utilization:** SNSEV is designed to work with real-time data collected from IoT devices, including sensors and cameras deployed in the urban landscape. These data, which include traffic conditions, roadblocks, and vehicle density, are crucial for dynamically adjusting routes and reducing response time. Most previous studies focus on static routing methods or rely on predefined conditions without incorporating real-time inputs.
- iii. **Prioritization of emergency vehicles:** Our algorithm not only focuses on efficient routing but also ensures that emergency vehicles receive priority in congested traffic environments. By utilizing real-time data, SNSEV identifies the optimal path that considers both traffic flow and the urgency of the emergency, a critical improvement over traditional routing algorithms which often fail to dynamically adapt to such priority needs.
- iv. **Scalability across urban environments:** We have demonstrated through simulations that SNSEV performs effectively across different scenarios, including variations in node density, mobility speeds, and network size. This scalability makes SNSEV applicable to a wide range of urban settings, from small towns to densely populated metropolitan areas, without significant degradation in performance. In comparison, many prior works have shown limited scalability or lack adaptability to varying network conditions.
- v. **Comparative advantage over existing protocols:** Extensive experiments were conducted to compare SNSEV with existing routing protocols (TVR, TBM, BA-CNN, R2P), and the results show that SNSEV consistently outperforms these in key metrics such as packet delivery ratio (PDR), routing overhead (RO), normalized routing load (NRL), average end-to-end delay (AETE), and throughput. Our method's ability to maintain low routing overhead and high throughput even under high mobility and node density situations marks a significant improvement over the existing strategies.
- vi. **Future integration potential:** Additionally, the architecture of SNSEV allows for future integration with other smart city technologies, including autonomous vehicles and intelligent transportation systems (ITSs). The potential to enhance emergency response capabilities through this integration further emphasizes the forward-looking nature of this research.

In summary, the novelty of SNSEV stems from its holistic approach to routing in smart cities, its use of fog and cloud computing for real-time optimization, its adaptability to real-world conditions, and its focus on prioritizing emergency vehicles. By addressing the challenges of dynamic urban environments with scalable and innovative techniques, our algorithm represents a significant advancement in the field of emergency vehicle navigation.

The paper is organized as follows: Sect. 2 provides a concise overview of previous studies. Section 3 outlines our proposed methodology, including its flowchart and algorithm. Section 4 analyzes the performance and assessment of the proposed approach. Section 5 summarizes the conclusions of the paper. Lastly, Sect. 6 highlights avenues for future research.

2 Related work

There have been numerous studies in recent years that focus on utilizing ICT-based solutions for traffic management in smart cities, particularly for emergency vehicles. In this section, we discuss some of the relevant literature on this topic.

The authors in [5] have created a dependable controller for a fast-paced environment and assessed how resistant these controllers are to various disruptions, such as accidents. The agent is given a comprehensive understanding of the environment by breaking down the intersection and modifying the state space. The algorithm developed can function independently of the accident's location or time, and the agent does not require retraining if the accident's location changes. The agent is taught using deep Q-learning and experience replay, and the model's efficiency is evaluated in the SUMO traffic micro-simulator. The simulation results prove that the proposed technique is effective at decreasing queues during disruptions.

In the previous study [6], the authors investigated the traffic signal control system of Shiraz City, which currently operates based on a fixed-time schedule and does not use intelligent methods. To improve the efficiency of the system, they utilized IoT and AI technologies to implement an intelligent traffic signal control system that employs multi-agent reinforcement learning (MARL) and real-time traffic data collected by sensors such as surveillance cameras. Their objective was to improve traffic flow at six signalized junctions in Shiraz City. We conducted numerical simulations for two synthetic intersections using simulated data and for a real-world map of Shiraz City using actual traffic data provided by the transportation and municipality traffic organization. They compared the results of our proposed approach with the traditional fixed-time scheduling system currently used in Shiraz. The simulation results demonstrated that their approach outperformed the fixed-time scheduling system in terms of average vehicle queue lengths and waiting times at intersections.

The authors of [7] utilized real-time traffic information to implement a deep reinforcement learning traffic light control system that prioritizes emergency vehicles at a crossroad. The system learns to adjust its policies to give priority to emergency vehicles over other vehicles, resulting in less waiting time and faster movement of cars. The results show that emergency vehicles experience the least delay when crossing the intersection. The study involved 200 to 600 cars and one emergency vehicle, and the use of reinforcement learning decreased the average delay by 2–16%. With more emergency vehicles, the average delay decreased by 27–40% for emergency vehicles.

In [8], a predictive controller based on a model was introduced to reduce traffic congestion by minimizing the number of cars waiting at red lights. Simulation models, as well as analytical models, can be utilized to examine how traffic systems respond under various circumstances. These models can imitate real-world processes and obtain information about them by using similar models of traffic flow. Traffic simulation models can be categorized as microscopic and macroscopic models. Microscopic simulation models take into account individual driver behavior as well as interactions with other vehicles and pedestrians, whereas macroscopic simulation models consider the entire traffic flow. A variety of microsimulation packages, such as AIMSUN, MATSim, Paramics, SUMO, and VISSIM, are used to study urban traffic dynamics.

AIMSUN, VISSIM, and SUMO are simulation models that enable users to create and control models through APIs using an external programming language. In traffic signal control, a microsimulation model of the network is employed to assess the current solution, and the outcomes of each simulation are given back to the proposed algorithm until a stopping criterion is met [9].

The ability to decrease the travel time of an Emergency Vehicle (EV) is crucial for enhancing the likelihood of the survival of victims. To avoid impromptu road blockages such as construction works, strikes, and accidents, an effective vehicle routing solution is necessary. The authors of [10] proposed an IoT network model for EV routing that incorporates a fuzzy logic-based data fusion technique. The data fusion process estimates the congestion at specific locations by considering sensory data and human inputs from the crowd. An open-source routing engine (open-source routing machine (OSRM)) then computes the shortest congestion-aware route by providing real-time traffic updates for a road network. Additionally, a sensor node has been designed to collect data on the emissions and speeds of moving vehicles on the road. An Android application is used to gather

information on road blockages from the crowd. With the aid of the Android application service, the driver of the EV is guided toward a medical center through the shortest congestion-aware route.

In [11] the authors proposed a reliable routing protocol (R2P) for vehicular ad-hoc networks in this study divides the network into overlapping zones. The master node (MN) of each zone maintains up-to-date routing boards for inter-/intra-zone connectivity. MNs maintain zone routing boards (ZRBs) while network nodes maintain private routing boards (PRBs). ERB, maintained by MN, registers gateways to neighboring zones, while ZRB and PRB register routes between zone nodes. R2P finds the most dependable path to the destination using a proprietary route discovery mechanism.

In [12] the authors offer efficient city ambulance vehicle routing. BA-CNN and bat algorithm underpin the method. It transfers patients discreetly, properly, and rapidly. This study employed ResNet CNNs [13–16]. Node technique creates city map. The control station informs the hospital and ambulance of the accident location. The driver provides the BA-CNN vehicle routing algorithm the ambulance's node position and accident site. The algorithm finds the driver's quickest path to the accident. The driver updates the algorithm with hospital and accident positions at the accident site. Then, the fastest route to the hospital is calculated. The bat method delivers offline data for any source–destination coordinate combination. Neural networks train offline data. Neural networks identify the shortest pathways between source and destination.

In [17], authors presented a trust-based model to detect rogue nodes in automotive networks. The proposed model first estimates node trust value and then identifies network rogue nodes. The authors route data through only trustworthy nodes. Network simulator (NS-2) simulates and evaluates the suggested model. The four performance matrices—successful packet delivery fraction, throughput, routing load, and end-to-end delay—assess network performance. The simulation reveals that the proposed model boosts network performance.

The study in [18] proposes trust-based authentication for clustered vehicle ad-hoc networks. Effective authentication should detect rogue nodes and reduce time and overhead. The approach creates trustworthy and stable clusters to stabilize the network. To pick cluster heads (CHs), the authors evaluate each vehicle's trust degree by combining the trust between cars and the trust between the vehicle and road side units (RSUs). Cluster Heads and verifiers watch vehicles. However, cluster heads offer a secure, optimum message transmission path. Each message has a trusted authority certificate since it is digitally signed by the sender, encrypted with a public/private key from a trusted authority (TA), and decoded by the recipient. This identifier authenticates the message sender and receiver. Simulations show that the suggested strategy improves malicious node detection, packet delivery, authentication delay, and overhead.

Table 1 summarizes the most common recent studies in several studies that propose and evaluate different intelligent traffic signal control systems using various techniques, such as deep Q-learning, multi-agent reinforcement learning (MARL), and fuzzy logic-based data fusion.

The research gaps in the previous algorithms are summarized in the following:

We may infer a possible research gap from the fact that each study tackles a specific aspect of traffic signal management, such as decreasing queues during disruptions, improving traffic flow in a particular city, prioritizing emergency vehicles, reducing travel time for emergency vehicles, and avoiding impromptu road blockages. Therefore, a potential research gap could be the lack of an integrated approach that addresses all of these aspects simultaneously to optimize urban traffic dynamics comprehensively.

The proposed SNSEV algorithm builds on these existing studies by utilizing fog and cloud computing technologies to provide real-time traffic management for emergency vehicles in smart cities. SNSEV collects real-time traffic data from IoT devices such as sensors and cameras, which is then processed and analyzed using fog computing technology. The cloud computing technology is then utilized to provide emergency vehicles with real-time navigation and priority control, reducing response time and ensuring that they reach their destination as quickly as possible [19]. The SNSEV algorithm has the potential to significantly improve the efficiency and effectiveness of emergency response in smart cities, leading to improved public safety and well-being.

Table 1 Recent studies in routing in smart cities

Reference	Simulator	Methodology	Results
[5]	SUMO traffic micro-simulator	Deep Q-learning and experience replay	Proposed technique effectively decreases queues during disruptions
[6]	Simulated data and actual traffic data provided by transportation and municipality traffic organization	Multi-agent reinforcement learning (MARL) and real-time traffic data collected by sensors such as surveillance cameras	Outperformed fixed-time scheduling system in terms of average vehicle queue lengths and waiting times at intersections
[7]	Real-time traffic information	Deep reinforcement learning traffic light control system	Emergency vehicles experienced the least delay when crossing the intersection, with average delay decreased by 2–16% for emergency vehicles
[8]	Simulation models (microscopic and macroscopic), AIMSUN, MATSim, Paramics, SUMO, and VISSIM	Predictive controller based on a model	Can imitate real-world processes and obtain information about them by using similar models of traffic flow
[9]	Microsimulation models of the network	Traffic signal control using a microsimulation model of the network	Used to assess the current solution, and the outcomes of each simulation are given back to the proposed algorithm until a stopping criterion is met
[10]	Fuzzy logic-based data fusion technique, open-source routing machine (OSRM), sensor node, Android application	IoT network model for EV routing	Driver of the EV is guided toward a medical center through the shortest congestion-aware route with real-time traffic updates provided for a road network
[11]	Network simulator (NS-2)	Divides the network into overlapping zones. The master node (MN) of each zone maintains up-to-date routing boards for inter/intra-zone connectivity	It improves the throughput, and decrease the bandwidth and delay time in experiments
[12]	MATLAB R2017a	The approach is based on the bat algorithm and convolutional neural network (BA-CNN)	The BA-CNN method is evaluated using end-to-end delay (EED), throughput, and packet delivery fraction (PDF)
[17]	Network simulator (NS-2)	First estimates node trust value and then identifies network rogue nodes	The model boosts network performance
[18]	Network simulator (NS-2)	Create trustworthy and stable clusters to stabilize the network	Improves malicious node detection, packet delivery, authentication delay, and overhead

3 SNSEV: a real-time navigation algorithm for emergency vehicles

Figure 1 depicts a system that involves fog and cloud computing layers, emergency vehicles, and a traffic light network. The image shows a city map with various roads and intersections, as well as the locations of emergency vehicles in different parts of the city. The fog and cloud computing layers are represented by a horizontal band across the top of the image. This layer is connected to the emergency vehicles through a series of arrows, indicating that data collected from the fog and cloud layer is being used to inform the movements of the emergency vehicles. The emergency vehicles are represented by small icons on the map, with different colors indicating different types of vehicles such as police cars, ambulances, and fire trucks. Arrows connect these icons to the traffic light network, indicating that the traffic lights are being used to direct the movement of the emergency vehicles through the city.

There is a network of traffic lights at intersections, showing how the lights change to allow emergency vehicles to move through quickly and safely. The traffic lights are connected to the fog and cloud computing layers through another set of arrows, illustrating how data from the cloud layer is used to inform the timing of the lights.

Overall, the figure provides a visual representation of a highly coordinated system that uses real-time data and technology to ensure the safe and efficient movement of emergency vehicles through a city. By leveraging the

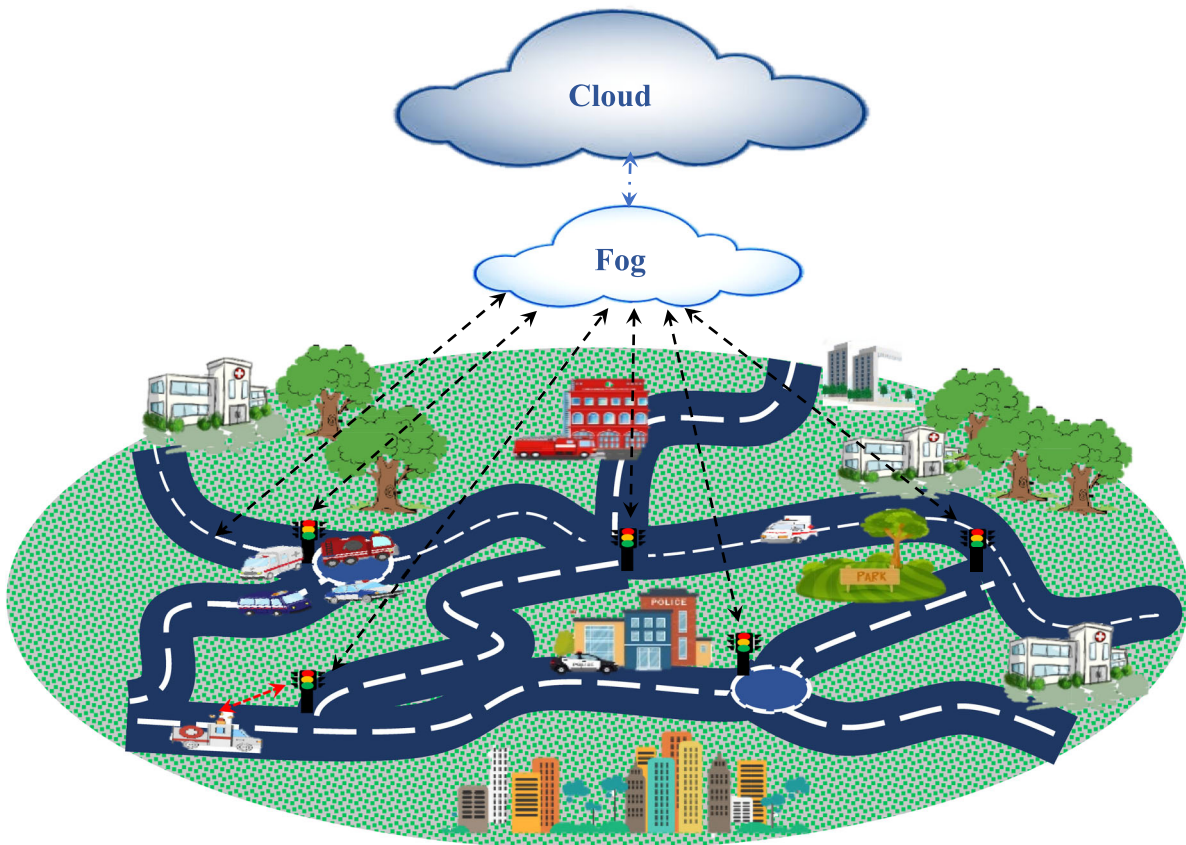


Fig. 1 Emergency vehicle routing system using fog and cloud computing layers

power of fog and cloud computing, emergency responders can be directed to the best route to their destination quickly and efficiently, while minimizing disruptions and delays to regular traffic flow.

The proposed SNSEV algorithm combines three main layers, namely (i) cloud computing, (ii) fog computing, and (iii) IoT, to improve emergency response in smart cities as illustrated in Fig. 1. The IoT layer collects real-time data using sensors and cameras, which is then processed and analyzed using fog computing technology. The fog layer utilizes this data to search for the destination and predict the shortest path using algorithms. It also manages traffic lights by predicting the predicted arrival time of emergency vehicles and adjusting the traffic lights accordingly.

The cloud computing layer provides real-time navigation and priority control to emergency vehicles. It utilizes the data from the fog layer and provides the shortest path to the destination, taking into account the current traffic conditions. This layer also controls the priority of the emergency vehicles, ensuring that they reach their destination as quickly as possible.

The proposed SNSEV algorithm is designed to improve the efficiency and effectiveness of emergency response in smart cities. It uses real-time data to predict the shortest path and adjusts to the current traffic conditions, reducing response time and ensuring that emergency vehicles reach their destination as quickly as possible.

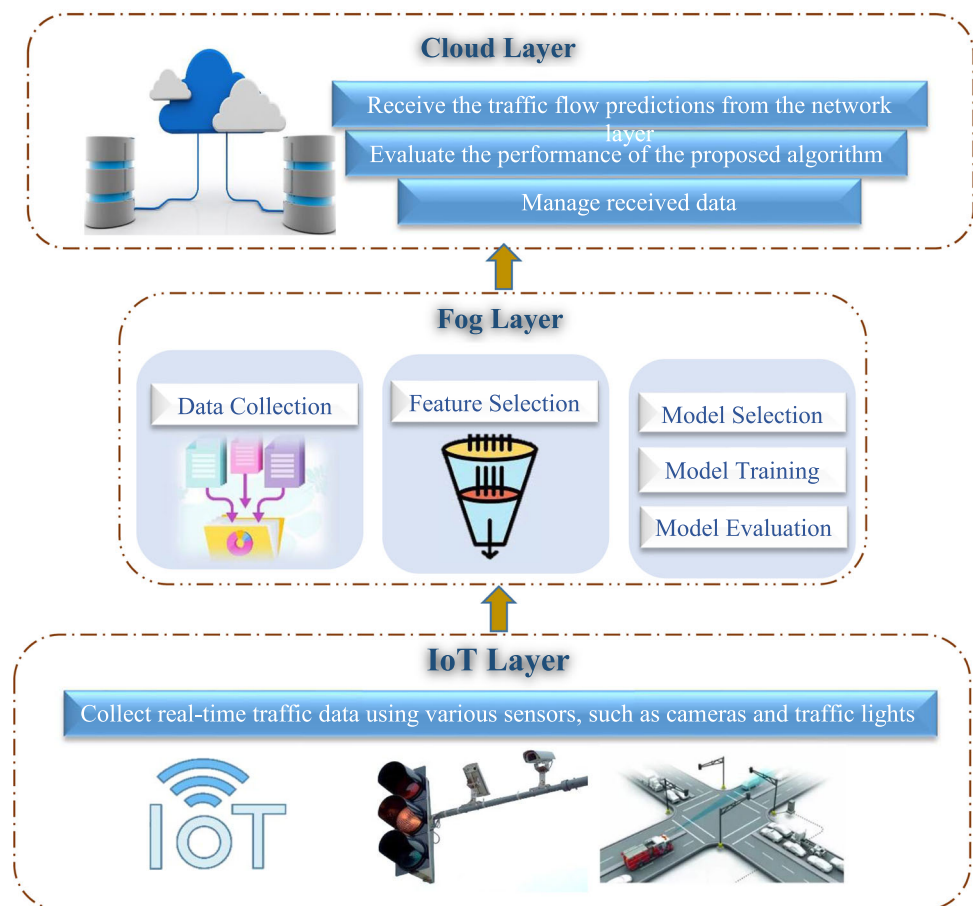
The SNSEV algorithm has the potential to significantly improve public safety and well-being by providing efficient and effective emergency response. However, there are challenges associated with the implementation of the algorithm, such as the cost of IoT devices and the need for collaboration between multiple agencies. Nonetheless, the benefits of SNSEV make it a promising solution for emergency response in smart cities (Fig. 2).

SNSEV is a real-time navigation algorithm designed for emergency vehicles in smart cities. It uses fog and cloud computing technology to manage traffic in real time. IoT devices such as sensors and cameras are utilized

to collect real-time traffic data, which are processed and analyzed using fog computing technology. Cloud computing technology is used to provide emergency vehicles with real-time navigation and priority control, reducing response time and ensuring they reach their destination as quickly as possible. The proposed SNSEV algorithm includes seven main phases which are:

- i. Emergency signal: An emergency vehicle sends a signal containing its id, location, and destination to the fog.
- ii. Search for destination: The fog receives the emergency signal and searches for the destination location in its maps. If the destination is not found in the fog maps, the fog sends a signal to the cloud to search for the destination in another fog. If the destination cannot be located, the algorithm stops.
- iii. Path determination: If the destination exists in the fog map, the fog determines the path from the location to the destination and sends it back to the emergency vehicle.
- iv. Predicted arrival time: Equations are applied to predict the arrival time for the emergency vehicle at each traffic light through the path to reach the destination, and a PAT (predicted arrival time) table is created.
- v. Periodic location update: The emergency vehicle updates its location periodically, and if the current time is not equal to a time in the PAT table, the fog recalculates the predicted arrival time for the rest of the traffic lights. If the emergency vehicle location is not equal to any traffic light location, the fog asks the emergency vehicle to send its location and recalculates the arrival time.
- vi. Traffic light management: If the predicted time equals the current time and the emergency vehicle location is not equal to any traffic light location, the fog asks the emergency vehicle to send its location. If the location is the same, the fog checks the traffic light state. If it is open, the emergency vehicle sends an

Fig. 2 Proposed SNSEV framework



acknowledge signal to the fog that it passed the traffic light. If it is closed, the fog sends a signal to the traffic light to open, and the emergency vehicle sends a passing acknowledge.

- vii. Destination reached: The fog checks the emergency vehicle location. If it is equal to the destination location, the algorithm stops. Otherwise, if the emergency vehicle has not reached the destination, the steps are repeated until the emergency vehicle reaches the destination.

3.1 Emergency signal

When an emergency vehicle needs to respond to an emergency situation, it sends an emergency signal to the fog computing system containing the vehicle's ID, current location, and destination. This signal is sent via a wireless communication system, such as cellular or satellite network. The fog computing system receives the signal and processes it to determine the best route for the emergency vehicle to take, taking into account real-time traffic data and any potential roadblocks. The steps of the emergency signal phase are illustrated in Algorithm 1.

Algorithm 1 Emergency signal

```
function emergency_signal (id, location, destination):
    // Send an emergency signal containing vehicle ID, current location, and destination to
    the fog
    signal = create_signal(id, location, destination)
    send_signal_to_fog(signal)

function create_signal(id, location, destination):
    // Create an emergency signal with vehicle ID, current location, and destination
    signal = {
        "vehicle_id": id,
        "location": location,
        "destination": destination
    }
    return signal

function send_signal_to_fog(signal):
    // Send the emergency signal to the fog for processing
    fog.receive_signal(signal)
```

3.2 Search for destination

After receiving the emergency signal from the EV, the fog will start searching for the destination location in its maps. The maps in the fog should cover the fog's area of responsibility. If the fog cannot find the destination location in its maps, the fog will send a signal to the cloud to search for the destination in another fog. The cloud will then send a response indicating whether the destination was found or not. If the destination cannot be located in any of the fogs, the algorithm will stop and the fog will send a message to the EV informing that the destination could not be located. The steps of the search for destination phase are illustrated in Algorithm 2.

Algorithm 2 Search for destination

```

• Input:
  ○ Emergency signal (vehicle id, vehicle location, vehicle destination)

• Steps:
  If vehicle destination is found in fog maps:
    // proceed with the algorithm
    proceed to the next phase
  else if vehicle destination is not found in fog maps:
    // forward the emergency signal to the other fog
    send a signal to the cloud to search for the destination in another fog
    if the destination is found:
      pass the emergency signal to that fog
      proceed to the next phase
    else:
      send a signal to the emergency vehicle that the destination is not correct
or cannot be located
      stop the algorithm

```

In summary, this phase ensures that the fog contains the destination location and allows the algorithm to proceed to the next phase of the navigation process. If the destination is not found, the algorithm tries to locate it in other fogs, and if that fails, it stops the algorithm.

3.3 Path determination

Once the fog finds the destination location in its maps, it calculates the path from the current location of the emergency vehicle to the destination. The path can be determined using various algorithms, such as Dijkstra's algorithm or A* algorithm. After calculating the path, the fog sends it back to the emergency vehicle to follow it as illustrated in Algorithm 3.

Algorithm 3 Path determination

```

• Steps:
  if destination is found in fog maps:
    path = calculate_path(current_location, destination)
    send_path_to_emergency_vehicle(path)
  else:
    stop_algorithm()

```

3.4 Predicted arrival time

Once the path from the emergency vehicle's location to the destination has been determined, the fog uses equations to predict the arrival time of the vehicle at each traffic light along the path. The fog takes into account the speed allowed on the roads through the path and the length of the path to estimate the time it will take for the vehicle to reach each traffic light. Based on this prediction, a PAT (predicted arrival time) table is created. The PAT table contains information about each traffic light along the path, including the traffic light's ID and the predicted time that the emergency vehicle will arrive at that traffic light.

This table will be used in the following phases to ensure that the emergency vehicle can pass through each traffic light without any delays and without disrupting the flow of other vehicles. The steps of the predicted arrival time phase are illustrated in Algorithm 4.

Algorithm 4 Predicted arrival time

```
• Steps:
  procedure createPATTable(path, speed_limit):
    PAT_table = empty_table()
    time = 0
    for each traffic_light in path:
      distance = distance_between(traffic_light, next_traffic_light)
      time += (distance / speed_limit)
      PAT_table[traffic_light.id] = time
    return PAT_table
```

In this algorithm, path is the determined path from the emergency vehicle's location to the destination, and speed_limit is the maximum allowed speed on the roads through the path. The algorithm iterates over each traffic light along the path and calculates the predicted time that the emergency vehicle will arrive at that traffic light, based on the distance between each traffic light and the speed limit on the roads between them. The predicted arrival time for each traffic light is stored in the PAT table, which is returned at the end of the procedure.

3.5 Periodic location update

In this phase, the emergency vehicle periodically updates its location, and the fog recalculates the predicted arrival time (PAT) for the rest of the traffic lights on the path to the destination. The fog checks if the current time is equal to a time in the PAT table and if not, it recalculates the PAT for the rest of the traffic lights. If the emergency vehicle location is not equal to any traffic light location, the fog asks the emergency vehicle to send its location and recalculates the PAT. The steps of the periodic location update phase are illustrated in Algorithm 5.

Algorithm 5 Periodic location update

```

• Steps:
  while (emergency vehicle is moving) {
    if (current time != time in PAT table) {
      // Recalculate predicted arrival time for the rest of the traffic lights
      for (each traffic light on the path) {
        calculate PAT for traffic light
      }
    }

    if (emergency vehicle location != any traffic light location) {
      // Ask emergency vehicle to send its location
      send location request to emergency vehicle

      // Recalculate predicted arrival time
      for (each traffic light on the path) {
        calculate PAT for traffic light
      }
    }

    // Wait for next location update
    wait for location update interval
  }

```

3.6 Traffic light management

In this phase, the algorithm manages the traffic lights to facilitate the passage of the emergency vehicle via set of steps as illustrated in Algorithm 6; (i) The fog checks if the predicted arrival time (PAT) of the emergency vehicle matches the current time. (ii) If the PAT matches the current time and the location of the emergency vehicle is not at a traffic light, the fog requests the location of the emergency vehicle. (iii) If the location of the emergency vehicle is the same as the previous location, the fog checks the state of the traffic light. (iv) If the traffic light is open, the emergency vehicle sends an acknowledge signal to the fog to confirm that it has passed the traffic light. (v) If the traffic light is closed, the fog sends a signal to the traffic light to open, and the emergency vehicle sends an acknowledge signal to confirm that it has passed the traffic light.

Algorithm 6 Periodic location update

```

• Steps:
  if predicted arrival time (PAT) matches current time:
    if emergency vehicle location is not at a traffic light:
      request emergency vehicle location
    if location is same as previous location:
      if traffic light is open:
        send acknowledge signal to fog
      else if traffic light is closed:
        send signal to open traffic light
        send acknowledge signal to fog

```

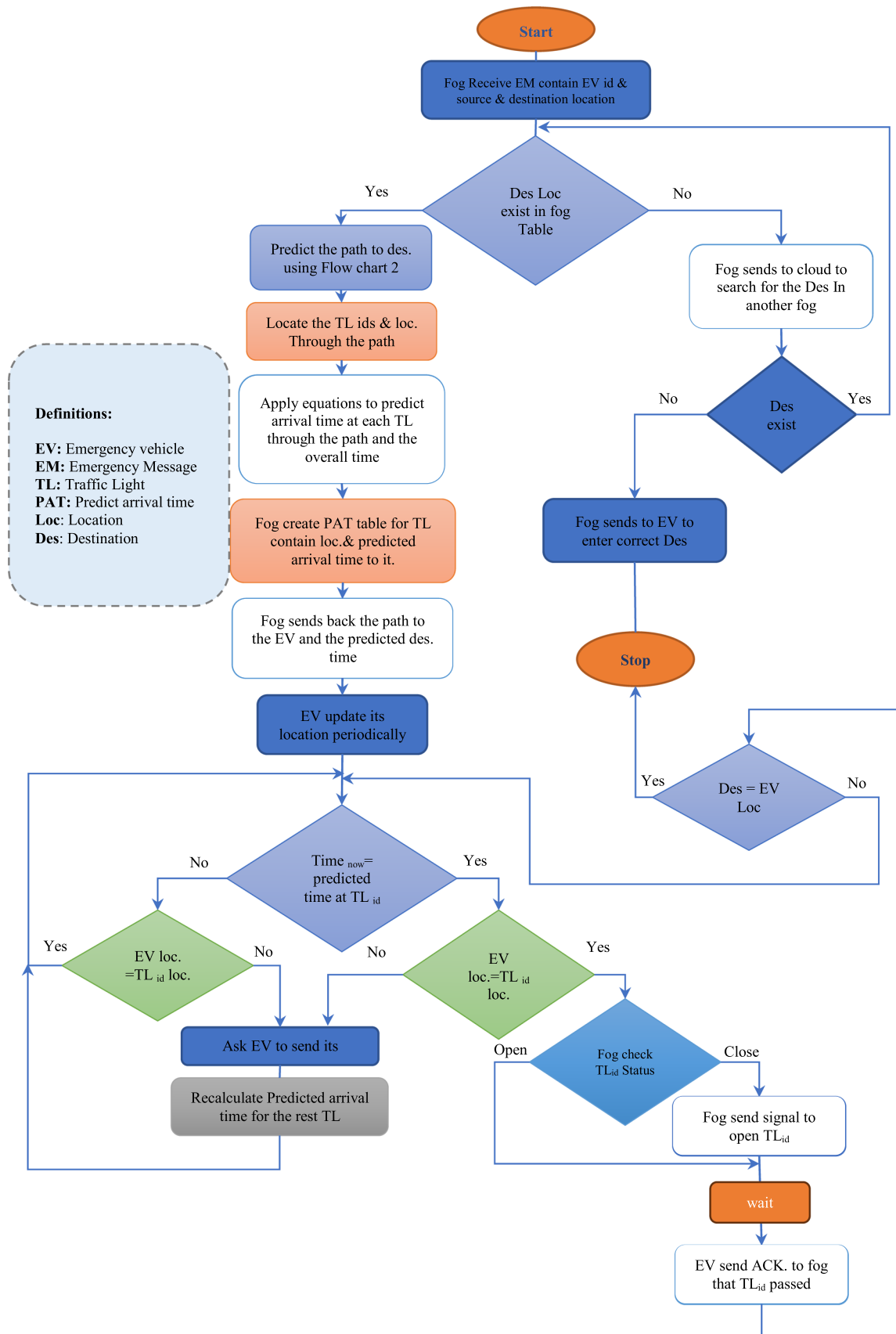


Fig. 3 Flowchart to predict shortest path

3.7 Destination reached

In this phase, the fog checks the location of the emergency vehicle to confirm if it has reached its destination. If the emergency vehicle has reached its destination, the algorithm stops. If the emergency vehicle has not yet reached its destination, the algorithm goes back to phase v and repeats the steps until the emergency vehicle reaches the destination. The steps of the destination reached phase are illustrated in Algorithm 7.

Algorithm 7 Destination reached

```

• Steps:
  while true:
    if emergency_vehicle_location == destination_location:
      stop_algorithm()
    else:
      go_to_phase_v()

```

The overall steps of the proposed SNSEV system are summarized in Fig. 3.

The measurements were carried out between key nodes involved in the EV routing and traffic management process. Specifically, the measurements focused on:

1. Emergency vehicle (EV) node—the source of real-time location and status updates.
2. Traffic control node—responsible for processing EV signals and adjusting traffic lights.
3. Roadside infrastructure nodes—these include sensors and communication units placed at intersections to relay traffic conditions.
4. Central management server—where data are aggregated and analyzed, and decision-making processes take place.

These nodes collectively facilitate the dynamic routing of emergency vehicles by predicting arrival times, optimizing traffic light adjustments, and ensuring smooth coordination within the network. The measurements were taken across these nodes to evaluate latency, communication efficiency, and response times.

The delay for each phase of the algorithm was measured and analyzed to assess the system's efficiency in real-time emergency response. The approximate delays for each phase are as follows:

1. Emergency vehicle (EV) signal transmission delay:
 - Time taken for the EV to send its location and emergency status.
 - Measured delay: ~ 50–100 ms (depending on network conditions).
2. Traffic control processing delay:
 - Time required to process the EV signal, predict arrival time, and adjust traffic lights accordingly.
 - Measured delay: ~ 100–200 ms.
3. Traffic light adjustment delay:
 - Time taken for the traffic control system to activate the signal change after receiving the command.
 - Measured delay: ~ 200–500 ms (varies based on traffic system infrastructure).

4. Vehicle wait time at traffic lights:

- Time required for the emergency vehicle to wait until the traffic light changes.
- Measured delay: $\sim 2\text{--}5$ s (depending on real-time traffic conditions and system responsiveness).

5. Navigation and obstacle avoidance delay:

- Time for the system to analyze real-time road conditions and adjust the vehicle's route accordingly.
- Measured delay: $\sim 100\text{--}300$ ms.

These delay estimates provide insights into system responsiveness and help refine optimization strategies for improved performance.

The position of the emergency vehicle (EV) is updated at regular intervals to ensure accurate tracking and real-time responsiveness. The update frequency depends on system configurations and network conditions.

Position update frequency

- Default update interval: Every 500 ms (0.5 s) to maintain real-time accuracy while minimizing network overhead.
- Adaptive update strategy: The system dynamically adjusts the update rate based on vehicle speed and network conditions:
- High-speed movement (above 60 km/h): Updates every 200–300 ms for precise tracking.
- Moderate speed (30–60 km/h): Updates every 400–500 ms for balanced efficiency.
- Low-speed movement (below 30 km/h) or waiting: Updates every 800–1000 ms to reduce unnecessary data transmissions.

This adaptive update mechanism ensures real-time tracking accuracy while optimizing communication efficiency.

The time required to determine the route depends on several factors, including the computational complexity of the algorithm, network conditions, and the size of the road network.

Route determination time

- Average processing time: The system typically determines the optimal route within 300–800 ms under normal conditions.
- Factors affecting processing time:
- Road network size: Larger networks may increase computation time.
- Real-time traffic data: Dynamic congestion updates may slightly extend processing.
- Computational resources: High-performance hardware reduces processing time.

By leveraging efficient path-planning algorithms and parallel processing, the system ensures minimal delay in route determination, allowing emergency vehicles to navigate efficiently.

The request to switch the traffic light is typically sent a few seconds before the emergency vehicle (EV) is predicted to reach the intersection. In our system, this lead time is dynamically calculated based on the vehicle's speed, distance to the intersection, and current traffic conditions. On average, the request is issued approximately 2–3 s prior to the EV's arrival. This buffer period allows the traffic light sufficient time to change state, ensuring that the EV can pass safely without unnecessary delays.

The location of the EV is determined using a combination of GPS and real-time communication with the traffic management system. When an emergency event is detected, the EV continuously transmits its GPS coordinates to the system. These coordinates are obtained via onboard GPS sensors and sent through a secure wireless network, such as 4G, 5G, or dedicated vehicular communication protocols (e.g., V2X). The traffic management system then processes this location data to predict the EV's estimated time of arrival (ETA) at various intersections,

enabling dynamic traffic light adjustments to ensure a clear route. Additionally, location data can be enhanced using sensor fusion techniques, integrating information from accelerometers, odometers, and city-wide surveillance networks for improved accuracy and reliability.

As illustrated in Fig. 3, the emergency vehicle sends a message to the fog contain its id, location, and destination. the fog contains google maps covering its area. The fog search in its maps for the destination, then sends it to the emergency vehicle to follow it. the fog creates PAT table (predicted arrival time) contain the traffic lights ids across the path and the predicted time the emergency vehicle will arrive to each traffic light on it. The fog will send a signal to each traffic light to open at the predicted time for the EV to reach it, and track the emergency vehicle location periodically until reach each destination without any conflict or affection on the movements of other vehicles.

At first, Emergency vehicle sends an emergency signal contain (vehicle id, vehicle location, vehicle destination). When Fog receives the emergency signal, it will search in its maps for the destination location. In case the destination not found in the fog maps, the fog will send a signal to the cloud to search for the destination in another fog, if it is found, it will pass the emergency signal to that fog. In case the destination not found in any other fog, the fog will send to the emergency vehicle that the destination is not correct or cannot be located and stop the algorithm. Otherwise, in case the destination exists in the fog map, the fog will get the path from location to destination and send it back to the emergency vehicle to follow it. Depending on the speed allowed on the roads through the path from location to the destination and the path length, equations will be applied to predict the arrival time for the emergency vehicle at each traffic light through the path to reach the destination and store it in a PAT (predicted arrival time) table.

The emergency vehicle updates its location periodically then check if the current time not equal to a time in PAT table, then fog check the emergency vehicle location if equal to TL_{id} . Recalculate the predicted arrival time for the rest traffic lights. In case the emergency vehicle location not equal to any traffic light location, ask the emergency vehicle to send its location and recalculate the arrival time. If the predicted time equal to the current time and the emergency vehicle location not equal to any TL_{id} location then fog ask the EV to send its location. in case location is the same, the fog checks the TL_{id} state, if its open the EV sends an acknowledge signal to the fog that it passes the TL_{id} . If the state of the TL_{id} is closed the fog sends signal to the TL_{id} to open then the EV sends a passing acknowledge.

Now fog check the EV location if its equal to the destination location, the algorithm will stop, otherwise if the EV not reach the destination, repeats the steps until EV reaches the destination.

After the signal is sent to adjust the traffic light in favor of EV, a wait command is required to ensure that the EV does not proceed until the traffic light has fully transitioned. This waiting mechanism is crucial for maintaining traffic safety and preventing potential collisions at intersections. The system continuously monitors the status of the traffic signal, and only after confirmation of the light change does the EV receive clearance to proceed. This ensures seamless coordination between traffic management systems and emergency response operations, enhancing both efficiency and safety.

To enhance the synchronization between EV movements and traffic light (TL) changes, we have introduced a **“Wait”** step in the system workflow. After the EV sends a request to open the TL, it must wait for confirmation that the light has successfully changed before proceeding. This ensures that the EV does not enter the intersection prematurely, reducing the risk of conflicts with other vehicles. The updated process improves traffic signal coordination, enhances safety, and ensures a smoother transition for emergency response vehicles navigating through intersections.

SNSEV is designed with several key strategies to ensure resilience in the face of infrastructure failures:

1. Local processing with fog computing: By leveraging fog computing, SNSEV processes data closer to the source, allowing emergency vehicles to continue functioning effectively even if the cloud layer becomes unreachable. This local processing enables real-time decision-making based on the most recent data available.

2. Redundancy and failover mechanisms: The system architecture includes redundancy measures to ensure that if one component of the fog or cloud infrastructure fails, alternative pathways or backup nodes can take over seamlessly. This approach helps maintain continuous service for emergency vehicles.
3. Periodic data synchronization: The fog layer periodically synchronizes data with the cloud to ensure that it has the most up-to-date information. In the event of a cloud failure, the fog layer can continue to operate with the last known data, allowing for ongoing navigation and traffic management.
4. Continuous monitoring: The system includes monitoring capabilities to detect potential failures in real-time. If a disruption is detected, SNSEV can automatically reroute traffic management tasks or adjust its operations to minimize any impact on emergency response.

By incorporating these strategies, SNSEV aims to maintain high reliability and performance, even in the face of disruptions in the fog or cloud infrastructure. We appreciate your interest in this critical aspect of system robustness.

The potential bottleneck of the proposed system lies in the real-time data processing and decision-making speed, particularly when multiple emergency vehicles (EVs) require priority access simultaneously. Another challenge is the coordination between traffic lights and vehicle movement, especially in high-density urban environments with unpredictable traffic patterns. Network latency in data transmission and the efficiency of communication between the EV, traffic control system, and other smart city components may also impact response time. However, we have implemented optimization techniques to mitigate these issues, ensuring smooth and efficient emergency vehicle routing.

These steps can be summarized as follows:

1. The emergency vehicle sends a signal to the fog with its ID, location, and destination.
2. The fog searches its maps for the destination, or if it's not found in the current fog, sends a signal to another fog or cloud service to search for it.
3. If the destination is found, the fog sends the path from the location to the destination back to the emergency vehicle, which can follow it.
4. The fog creates a predicted arrival time (PAT) table for each traffic light along the path and predicts when the emergency vehicle will arrive at each traffic light.
5. The fog sends a signal to each traffic light to open at the predicted time for the emergency vehicle to pass through.
6. The fog periodically tracks the emergency vehicle's location and updates the PAT table accordingly.
7. If the emergency vehicle reaches a traffic light and it is open, it sends an acknowledgment signal to the fog. If it is closed, the fog sends a signal to open it before the emergency vehicle passes through.
8. Once the emergency vehicle reaches the destination, the algorithm stops. If it has not yet reached the destination, the steps are repeated until it does.

4 Implementation and experiments

In this section, the proposed routing strategy will be compared to the most recent routing strategies TVR [18], TBM [17], BA-CNN [12], R2P [11], and our proposed [SNSEV] using the metrics packet delivery ratio (PDR), routing overhead (RO), normalized routing load (NRL), average end-to-end delay (second), and throughput which are defined in Table 2.

Table 2 Performance metrics

Metrics	Description
Packet delivery ratio (PDR)	It is the proportion of data fragments delivered to destinations by the sources. PDR measures the number of packets that a protocol can successfully convey. PDF can be determined by dividing the number of packets received by the destination by the number of packets sent by the source. PDF can be calculated as follows: $PDR = (P_p/PT) \times 100$, where P_p is the number of data packets that can reach the destination node and PT is the total number of data packets delivered by the source during the simulation time
Routing overhead (RO)	It represents the total number of control or routing packets generated by the routing protocol throughout the simulation. Consider routing overhead for all packets sent or forwarded at the network layer
Normalized routing load (NRL)	Amount of routing messages “transmitted” for each data packet “delivered” at the destination. NRL therefore equals routing packet/received packets
Average end-to-end delay (second)	This includes any delay caused by buffering during route discovery latency, interface queueing, retransmission delay at the MAC, propagation, and transfer time. It is the time required for a data payload to be transmitted from source to destination across a MANET. Consequently, $D = (T_r - T_s)$, where T_r receives Time and T_s sends Time
Throughput	The throughput of a routing protocol is the total number of effective packets received by all destination nodes in a given amount of time. The unit of transmission is megabits per second (MB/s), but in our simulation, we used Kilobits per second (kb/s)

Table 3 NS-2 simulation parameters

Parameter	Value
Platform	Linux Ubuntu 14.04
NS version	NS 2.33
Traffic type	CBR
Radio propagation	Two-ray ground
Antenna type	Omni-Antenna
Simulation time	10 s
Pause time	5, 10, 20, 40, 100
Number of nodes	25, 50, 75, 100
Network size	1200*1200 m

4.1 Experimental setup

Experiments are conducted utilizing NS-2 application. Table 3 summarizes the experimental environment. To comprehensively evaluate the performance of the proposed SNSEV algorithm, we conducted a series of simulations under various scenarios that reflect real-world conditions encountered by emergency vehicles. The following scenarios were tested:

1. Varying mobility speeds: Different mobility speeds were simulated, ranging from stationary to high speeds. This variation helps assess how well each routing algorithm can adapt to dynamic conditions, which is crucial for emergency response situations where vehicles may need to navigate through fluctuating traffic.
2. Pause times: We examined several pause times (5, 10, 20, 40, and 100 s) to evaluate the impact of node mobility on packet delivery and overall system performance. These scenarios simulate various traffic conditions, allowing us to analyze how effectively the algorithms maintain communication under different levels of vehicle activity.
3. Node density: The simulations included varying numbers of nodes (25, 50, 75, and 100) to test the scalability of each algorithm. This aspect is particularly relevant for urban environments, where higher vehicle density can significantly affect the performance of routing strategies.
4. Network size: All simulations were conducted within a controlled environment of 1200×1200 m. This size was chosen to reflect realistic urban settings while allowing for clear analysis of the routing performance.

5. **Traffic patterns:** We utilized constant bit rate (CBR) traffic to simulate data transmission typical of emergency vehicles. This approach helps in understanding how well each algorithm can handle consistent data loads during operations.

4.2 PDR (packet delivery ratio)

In this investigation, the packet delivery ratio for TVR, TBM, BA-CNN, R2P, and the proposed SNSEV was determined in relation to pause times and mobility speeds. Figure 4 depicts TVR versus various mobility velocities using 40 sources and a pause time of 20 s.

As depicted in Fig. 4, all protocols deliver the vast majority of data packets (between 90 and 96 percent). In contrast, SNSEV outperforms all other algorithm. Because the algorithm updates the routing tables of nodes with current routing information. As depicted in Fig. 5, when a lengthy pause time indicates low mobility, the performance of the five protocols is comparable. They are able to transfer the majority of packets due to fewer network topology changes. Small halt time with increasing mobility, SNSEV outperforms both R2P, BA-CNN, TBM, TVR in every circumstance.

4.3 Routing overhead (RO)

Figure 6 compares the routing overheads of various routing protocols with the optimum mobility speed when using 40 resources, respectively. Increasing the mobility pace of nodes is typically associated with an increase in routing overhead. As shown in Fig. 5, TVR has a lower routing overhead than TBM when node mobility is increased. the performance of the five routing protocols is comparable despite mobility. Moreover, TVR utilizes routing currency, so it has a lower routing overhead than TBM, which is used to reduce flooding. TVR, on the other hand, does not utilize periodic routing advertisement messages, but rather employs artificial snacks, thereby reducing the network bandwidth penalty. Therefore, it incurs fewer routing overheads than both TBM and BA-CNN. As depicted in Fig. 5, SNES outperforms all other routing protocols because it incurs the least amount of

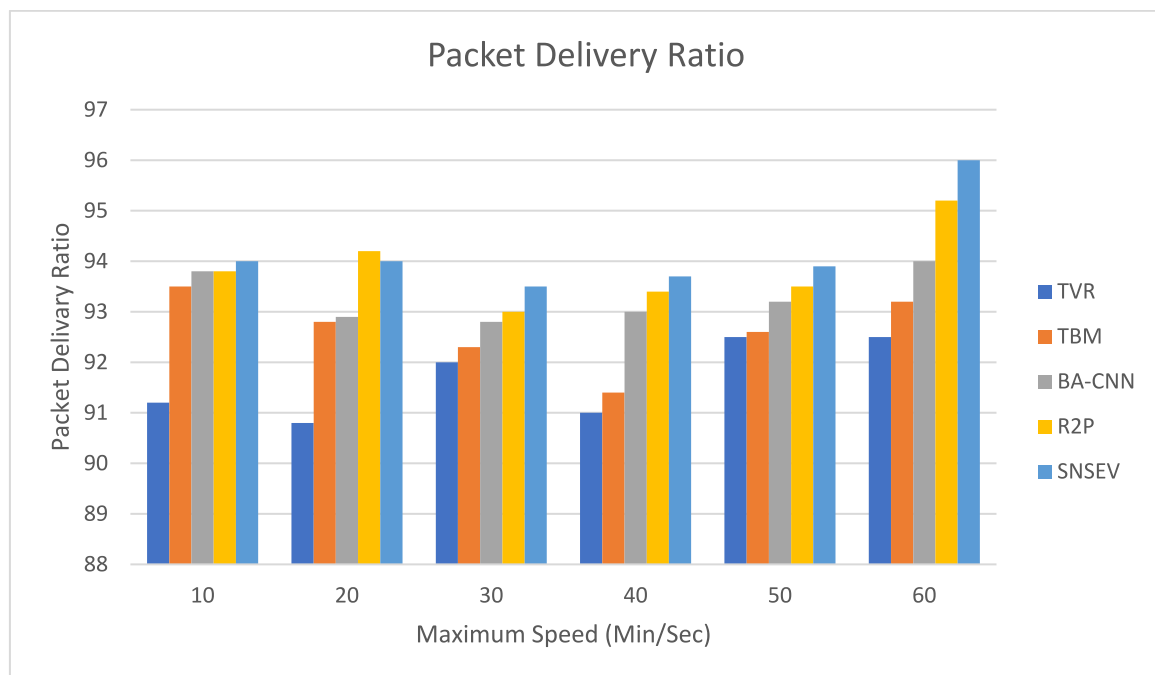


Fig. 4 Packet delivery ratio against different algorithms

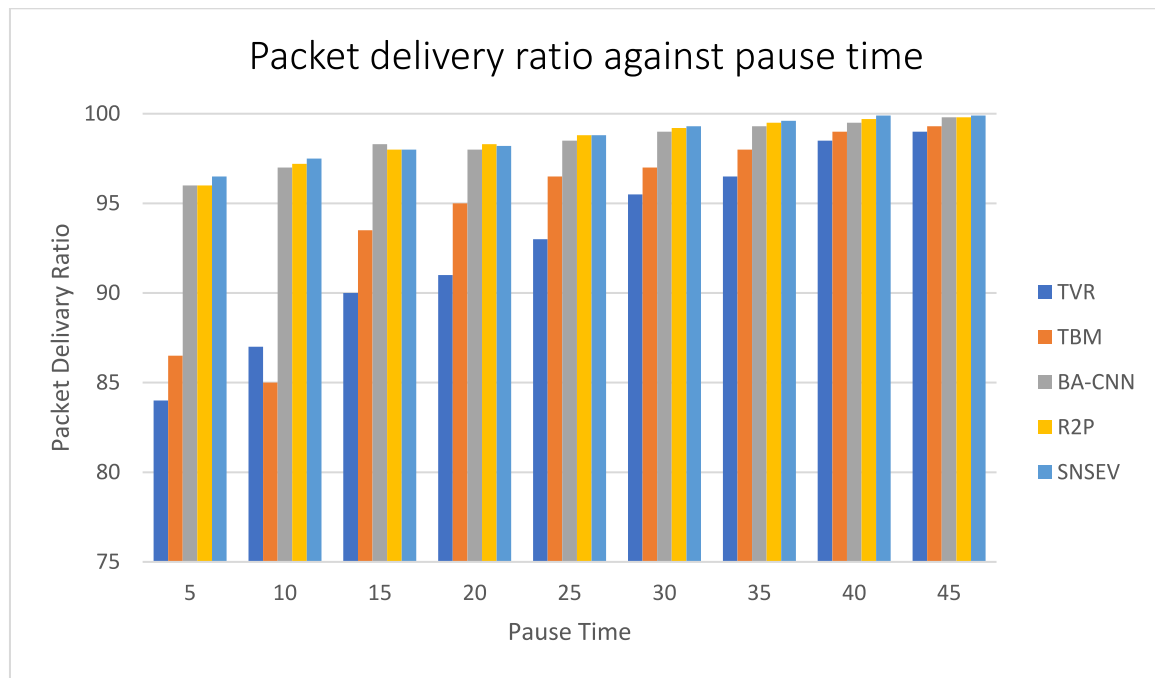


Fig. 5 Packet delivery ratio against pause time

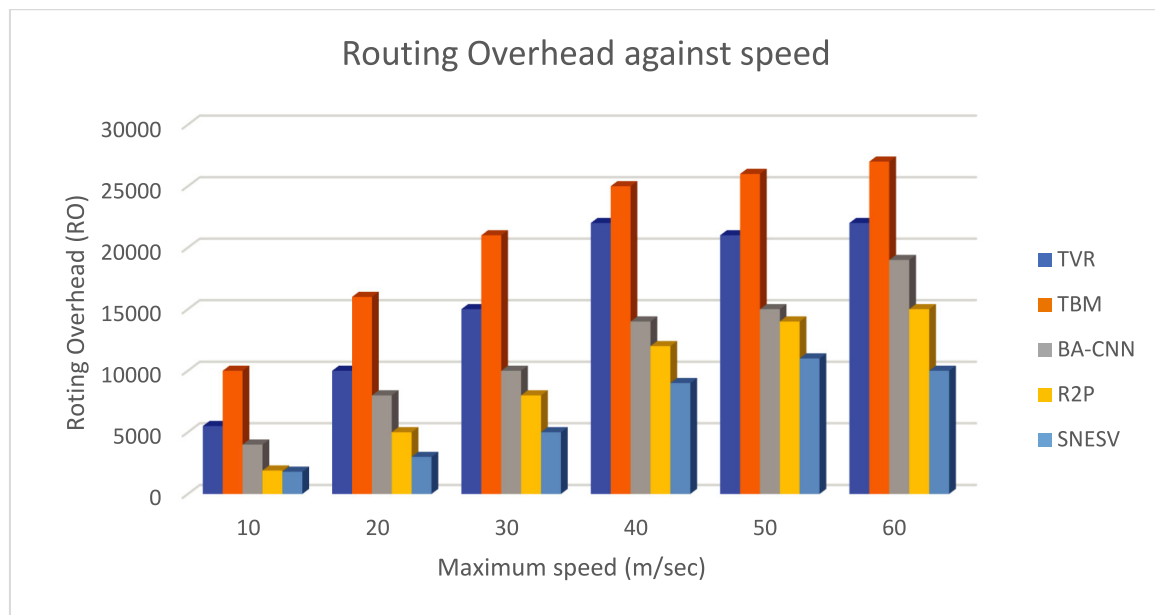


Fig. 6 The routing overhead against the maximum speed

routing overhead. This occurred as a result of its continuous monitoring of network topology modifications. In addition, the most reliable route is frequently chosen for data transmission, which reduces routing overheads accordingly.

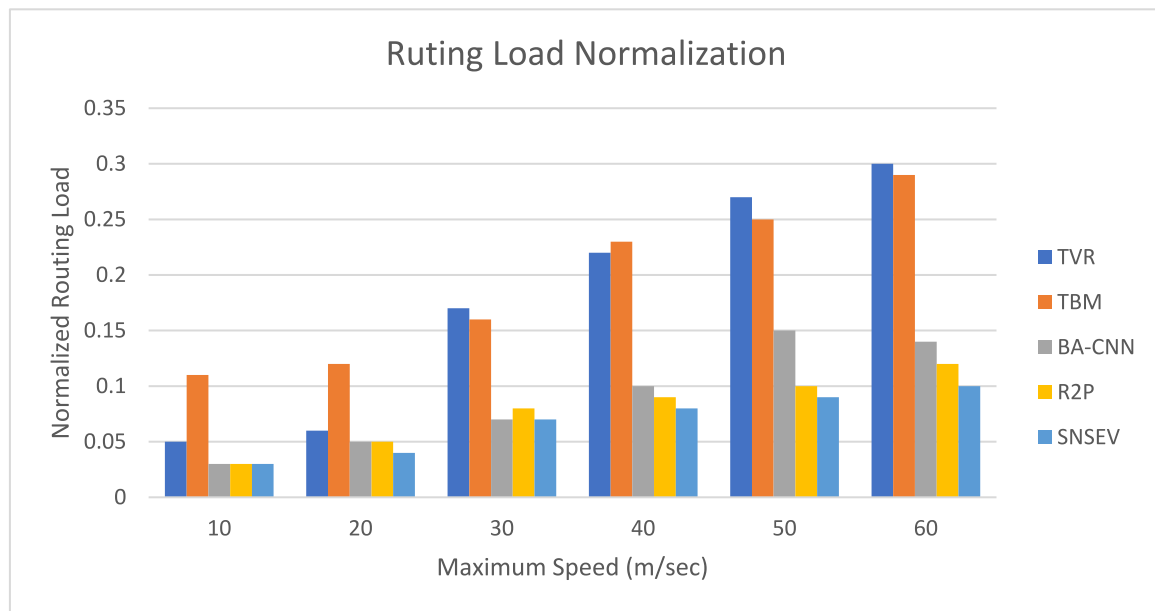


Fig. 7 The normalized routing load against the maximum speed

4.4 Routing load normalization (NRL)

As shown in Fig. 7, SNES's behavior in terms of normalized routing burden is unaffected by changes in source speed and number. In the majority of cases, it outperforms TVR, TBM, BA-CNN, and R2P. However, when mobility increases, TBM generally performs better than TVR. SNES has a reduced NRL compared to the other protocols. This is due to the fact that it selects the most reliable data transmission route. As a result, minimal route maintenance is necessary, reducing the normalized routing burden.

4.5 The average end-to-end delay (AETE)

As shown in Fig. 8, TVR has a significantly longer delay than TBM, BA-CNN, R2P, and SNES. In all instances of real-time packet delivery, SNES is the optimal solution. In contrast, TVR results in greater delay due to route caching. Although TVR can respond to a route rapidly, when a route is rebuilt, it experiences a lengthy delay.

4.6 Throughput

Throughput for TVR, TBM, BA-CNN, R2P, and SNES is monitored in this experiment with varying numbers of source nodes, and an average is then determined. Figure 9 shows that the suggested SNES method has a better ability to transmit nearly all data packets during the test period than TVR, TBM, BA-CNN and R2P. While SNES has a fast response time, when a route is being rebuilt, it takes a very lengthy time.

5 Conclusion

In this paper, we have presented SNSEV, a real-time navigation algorithm for emergency vehicles in smart cities. Our algorithm utilizes fog and cloud computing technology for real-time traffic management, improving response time and ensuring that emergency vehicles reach their destination as quickly as possible. By utilizing IoT devices such as sensors and cameras, we collect real-time data on traffic conditions and roadblocks, which is then

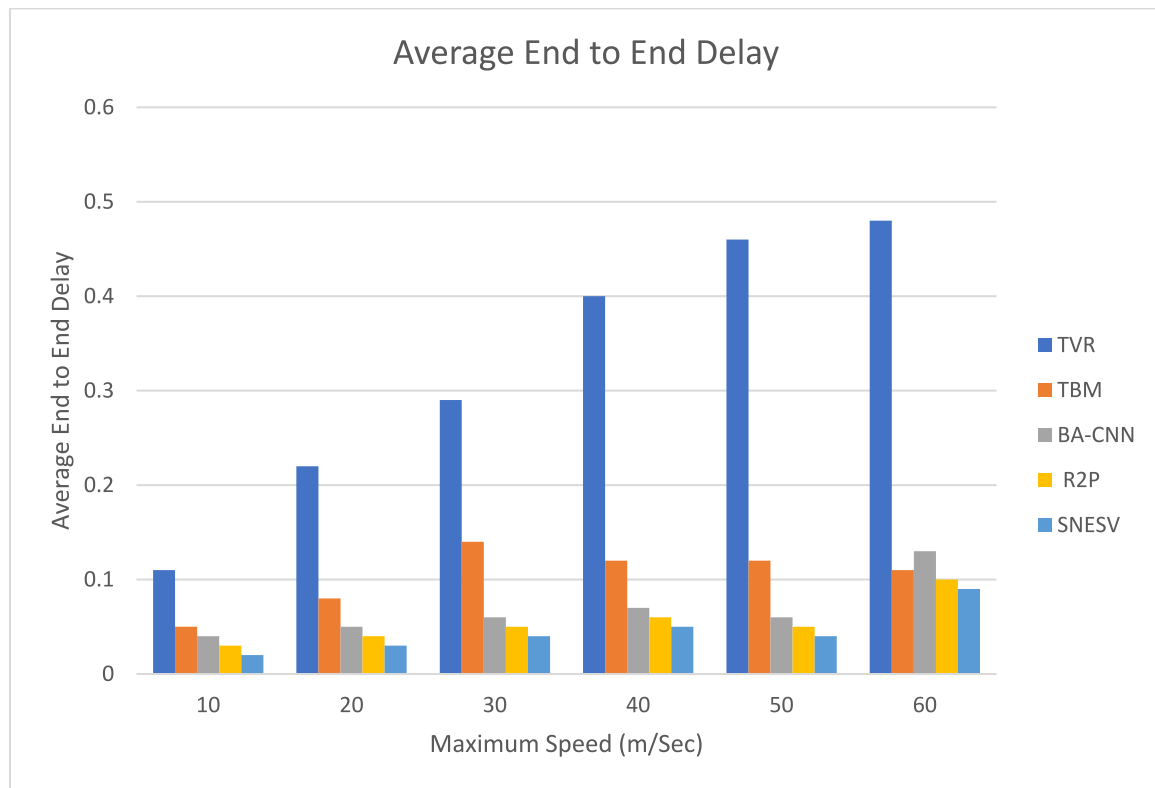


Fig. 8 The average end-to-end delay against the maximum speed

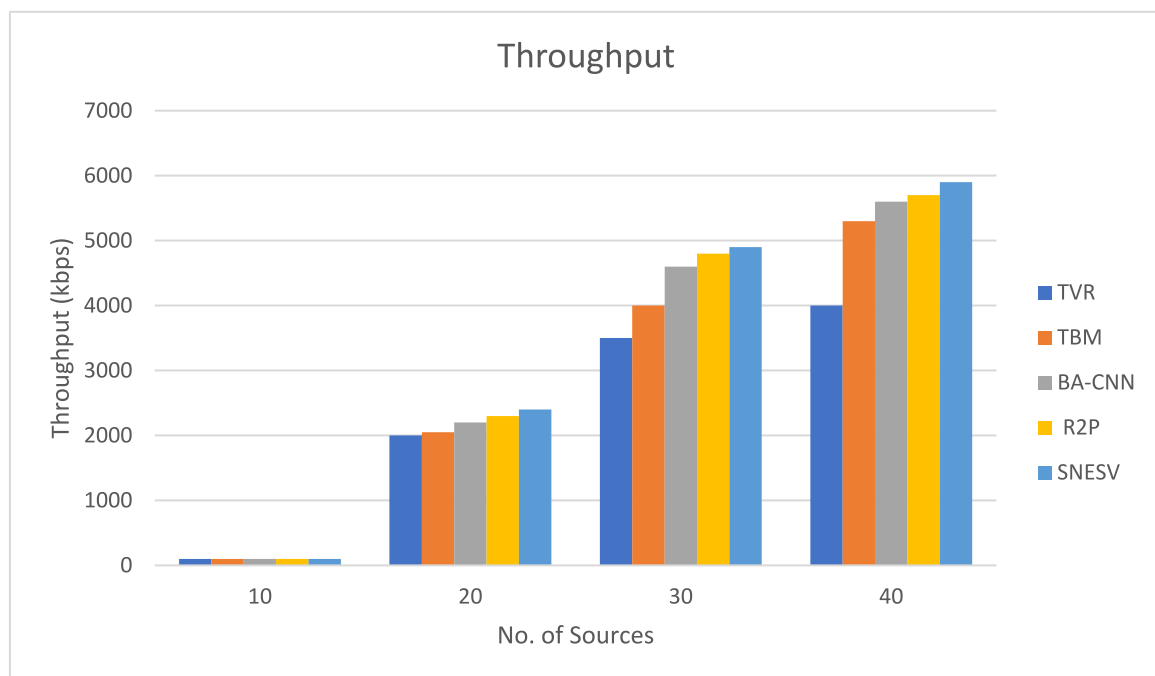


Fig. 9 The throughput against no. of sources

processed and analyzed using fog computing technology. The cloud computing technology is then used to provide real-time navigation and priority control to emergency vehicles. The proposed SNSEV algorithm has the potential to significantly improve the efficiency and effectiveness of emergency response in a smart city. It can

reduce response time and improve public safety and wellbeing. However, our algorithm also presents some challenges, such as the need for a robust and reliable fog and cloud computing infrastructure, the potential for privacy and security concerns, and the need for ongoing maintenance and updates.

6 Future work

In future work, we plan to further evaluate the effectiveness of SNSEV by conducting field tests in real-world emergency situations to gather empirical data and refine our algorithm based on practical insights. We envision integrating SNSEV with other smart city technologies to enhance emergency response capabilities. This includes collaboration with autonomous vehicle systems, allowing for dynamic routing of emergency vehicles and clearing paths for responders. Additionally, integrating SNSEV with intelligent transportation systems (ITSs) could facilitate better coordination between traffic signals and emergency vehicle movements, prioritizing their passage through intersections. We also aim to establish a data-sharing framework with other smart city services, such as public transport and city surveillance, to enhance situational awareness and promote coordinated responses during emergencies. Future iterations of SNSEV may leverage machine learning techniques, such as OCNN [20–22] and ResNet [23], to improve predictive analytics and traffic pattern recognition. Moreover, employing ERNN for stress detection could facilitate adaptive response strategies, while implementing attention mechanisms could help prioritize critical data inputs [24]. Utilizing correlation algorithms may refine the decision-making process for emergency routing [25, 26], and integrating YOLOv8 [27] for real-time obstacle detection could enhance navigation accuracy. Overall, we believe that by expanding SNSEV's integration with various technologies, we can significantly improve public safety and responsiveness in emergencies [28]. The study by Talaat and Gamel proposed A2M-LEUK, an attention-augmented algorithm designed for effective blood cancer detection in children using deep learning techniques.

Authors' contributions Equivalent roles.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB). The authors received no specific funding for this study.

Data availability Private data.

Declarations

Conflict of interest There is no conflict of interest. The authors declare that they have no conflicts of interest to report regarding the present study.

Ethical approval There is no any ethical conflicts.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. undesa_pd_2022_global_population_growth.pdf

2. Aleko DR, Djahel S (2020) An efficient adaptive traffic light control system for urban road traffic congestion reduction in smart cities. *Information* 11(2):119. <https://doi.org/10.3390/info11020119>
3. 1708.07188.pdf (arxiv.org)
4. Oladimeji D, Gupta K, Kose NA, Gundogan K, Ge L, Liang F (2023) Smart transportation: an overview of technologies and applications. *Sensors* 23(8):3880. <https://doi.org/10.3390/s23083880>
5. Zeinaly Z, Sojoodi M, Bolouki S (2023) A resilient intelligent traffic signal control scheme for accident scenario at intersections via deep reinforcement learning. *Sustainability* 15(2):1329. <https://doi.org/10.3390/su15021329>
6. Damadam S, Zourbakhsh M, Javidan R, Faroughi A (2022) An intelligent IoT based traffic light management system: deep reinforcement learning. *Smart Cities* 5(4):1293–1311. <https://doi.org/10.3390/smartcities5040066>
7. Shamsi M, Kenari AR, Aghamohammadi R (2022) Reinforcement learning for traffic light control with emphasis on emergency vehicles. *J Supercomput.* <https://doi.org/10.1007/s11227-021-04068-w>
8. Jafari S, Shahbazi Z, Byun Y-C (2021) Improving the performance of single-intersection urban traffic networks based on a model predictive controller. *Sustainability* 13:5630
9. Qadri SSSM, Gökçe MA, Öner E (2020) State-of-art review of traffic signal control methods: challenges and opportunities. *Eur Transp Res Rev* 12:55
10. Vemireddy S, Raul S (2020) Fuzzy logic-based emergency vehicle routing: an IoT system development for smart city applications. *Comput Electric Eng.* <https://doi.org/10.1016/j.compeleceng.2020.106839>
11. Saleh AI, Gamel SA, Abo-Al-Ez KM (2017) A reliable routing protocol for vehicular ad hoc networks. *Comput Electr Eng* 64:473–495
12. DarwasshHanawy Hussein T et al (2022) BA-CNN: Bat algorithm-based convolutional neural network algorithm for ambulance vehicle routing in smart cities.”. *Mobile Inf Syst* 2022:7339647
13. El-Rashidy N, ElSayed NE, El-Ghamry A, Talaat FM (2022) Prediction of gestational diabetes based on explainable deep learning and fog computing. *Soft Comput* 26(21):11435–11450
14. Hanaa S, Fatma BT (2022) Brain tumor detection and classification using deep learning and sine-cosine fitness grey wolf optimization. *Bioengineering* 10(1):18. <https://doi.org/10.3390/bioengineering10010018>
15. Talaat FM (2023) Real-time facial emotion recognition system among children with autism based on deep learning and IoT. *Neural Comput Appl.* <https://doi.org/10.1007/s00521-023-08372-9>
16. Talaat FM (2023) Crop yield prediction algorithm (CYPA) in precision agriculture based on IoT techniques and climate changes. *Neural Comput Appl.* <https://doi.org/10.1007/s00521-023-08619-5>
17. Tripathi KN, Sharma SC (2020) A trust based model (TBM) to detect rogue nodes in vehicular ad-hoc networks (VANETS). *Int J Syst Assur Eng Manag* 11:426–440
18. Mirsadeghi F, Rafsanjani MK, Gupta BB (2021) A trust infrastructure based authentication method for clustered vehicular ad hoc networks. *Peer-to-Peer Netw Appl* 14:2537–2553
19. Talaat FM, Alshathri S, Nasr AA (2022) A new reliable system for managing virtualcloud network. *Comput Mater Continua* 73(3):5863–5885. <https://doi.org/10.32604/cmc.2022.026547>
20. El-Rashidy N, Ebrahim N, el Ghamry A, Talaat FM (2022) Utilizing fog computing and explainable deep learning techniques for gestational diabetes prediction. *Neural Comput Appl.* <https://doi.org/10.1007/s00521-022-08007-5>
21. Hassan E, Elmougy S, Ibraheem MR, Hossain MS, AlMutib K, Ghoneim A, AlQahtani SA, Talaat FM (2023) Enhanced deep learning model for classification of retinal optical coherencetomography images. *Sensors* 23:5393. <https://doi.org/10.3390/s23125393>
22. Talaat FM, Gamel SA (2023) Machine learning in detection and classification of leukemia using C-NMC_Leukemia. *Multimed Tools Appl.* <https://doi.org/10.1007/s11042-023-15923-8>
23. Hassan E, El-Rashidy N, Talaat FM (2022) Review: mask R-CNN models. *Nile J Commun Comput Sci.* <https://doi.org/10.21608/njccs.2022.280047>
24. Siam AI, Gamel SA, Talaat FM (2023) Automatic stress detection in car drivers based on non-invasive physiological signals using machine learning techniques. *Neural Comput Appl.* <https://doi.org/10.1007/s00521-023-08428-w>
25. Talaat FM, Gamel SA (2023) A2M-LEUK: attention-augmented algorithm for blood cancer detection in children. *Neural Comput Appl.* <https://doi.org/10.1007/s00521-023-08678-8>
26. Gamel SA, Hassan E, El-Rashidy N et al (2023) Exploring the effects of pandemics on transportation through correlations and deep learning techniques. *Multimed Tools Appl.* <https://doi.org/10.1007/s11042-023-15803-1>
27. Talaat FM, ZainEldin H (2023) An improved fire detection approach based on YOLO-v8 for smart cities. *Neural Comput Appl.* <https://doi.org/10.1007/s00521-023-08809-1>
28. Talaat FM (2022) Route planning for autonomous mobile robots using a reinforcement learning algorithm. *Actuators* 12(1):12. <https://doi.org/10.3390/act12010012>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Fatma M. Talaat^{1,2}  · **Samah A. Gamel**³

✉ Fatma M. Talaat
fatma.nada@ai.kfs.edu.eg

Samah A. Gamel
s.adel.gamel@gmail.com

¹ Faculty of Artificial Intelligence, Kafrelsheikh University, Kafrelsheikh 33516, Egypt

² Faculty of Computer Science & Engineering, New Mansoura University, Gamasa 35712, Egypt

³ Faculty of Engineering, Horus University, Damietta, Egypt